

访谈笔记

访谈笔记：赵晨阳谈 SGLang、退学潮、AI Coding 与开源社区

基于 Ungrounded 不着边际 EP02 整理

2026-05-08



视频作者/频道：Ungrounded 不着边际

发布日期：2026-05-06

视频时长：01:07:50

视频链接：<https://www.bilibili.com/video/BV1oRRyBREPU/>

项目主页：<https://github.com/hqh1025/ai-course-notes>

目录

1 SGLang 的位置：AI 五层架构里的中间层	2
1.1 本章小结	3
2 退学潮：不是反学校，而是 applied research 被迫面对市场	3
2.1 本章小结	4
3 AI 时代的 Research Taste：人类剩下的是抽象层级	4
3.1 本章小结	5
4 AI Coding：代码便宜之后，责任变贵了	5
4.1 本章小结	6
5 AI 是放大器：从学习反馈到认知卸载	6
5.1 本章小结	6
6 Agent 不会 Say No：人类的决策边界	7
6.1 本章小结	7
7 开源社区：AI 时代如何记录贡献与维持秩序	7
7.1 本章小结	8
8 总结：AI 降低执行成本，也提高判断成本	8

导言

这期访谈表面上是在聊赵晨阳：清华本科，UCLA 读博十五个月，参与 SGLang，从开源项目走向推理引擎创业；视频发布时，RadixArk 刚官宣一亿美元种子轮融资。若只按新闻点读，很容易把它理解成“硅谷退学潮里的年轻人故事”。但整场真正值得保存的部分，不是退学本身，而是退学、SGLang、AI Coding 和开源社区共同指向的一个更深问题：当 AI 把执行成本迅速压低之后，人类还剩哪些职责必须亲自承担？

赵晨阳反复强调“做事要本质一点”。这句话贯穿全场。他谈学术圈时，批评的是拿着锤子找钉子、虚空 formulate 一个 evaluation 的惯性；谈退学时，关心的是 applied research 到底需不需要真正接受市场和工程验证；谈 AI Coding 时，强调的是代码生产变便宜后，人类更需要负责 design、review、say no 和承担后果；谈开源社区时，真正关心的也不是“贡献数量”，而是一个项目如何在 AI 生成代码泛滥后仍然维持责任边界、审美和协作秩序。

因此，这份笔记不按时间顺序复述，而按六个主题重组：推理引擎的生态位置、退学潮背后的研究范式变化、AI 时代的 research taste、AI Coding 对代码治理的冲击、AI 作为放大器带来的认知风险，以及开源社区如何在生成式代码时代重新定义贡献。

阅读这份笔记的三条主线

第一条主线是“本质性”：applied research 要有 application，fundamental research 要有自治的问题体系，不能只在虚构靶子上优化。第二条主线是“责任边界”：AI 让执行更便宜，但也让不动脑子的 PR、代码和内容更容易泛滥。第三条主线是“社区与组织”：开源项目不只是代码仓库，而是要记录贡献、筛选责任、传递信息并组织人一起做事。

原视频没有官方字幕，本整理基于本地 Whisper 转写和逐段校读完成。正文不做逐字稿，而是尽量提炼访谈中的判断链条和可迁移观点。

1 SGLang 的位置：AI 五层架构里的中间层

赵晨阳开场先介绍自己：清华计算机本科，2024 年毕业，在 UCLA 读了十五个月博士，随后在 2025 年 11 月左右开始创业，加入推理引擎方向的公司。主持人随即把问题引向 SGLang：它如何从开源组织走向公司化，赵晨阳在其中有什么体会。

他先解释什么是推理引擎。普通用户接触 AI，看到的是 ChatGPT、Claude、AI PPT、Sora、AI Coding 等前端产品；但用户请求发到后端之后，服务器如何调度模型、如何管理 KV cache、如何量化、如何处理不同模型形态、如何提高吞吐和降低延迟，这才是推理引擎要解决的问题。Hugging Face 的 `model.generate` 可以帮助人理解最朴素的生成过程，但真正服务线上请求，需要大量系统层优化。

赵晨阳借用一个五层架构来定位推理：最上层是 Agent 和应用；下面是具体模型，如 Claude、GPT、开源模型；再往下是推理引擎，如 OpenAI internal engine、TensorRT-LLM、SGLang；再往下是硬件，再往下是半导体、电力等基础设施。越往上变化越快，越往下周期越慢；推理引擎恰好处在中间，是模型厂商和硬件厂商之间的关键承接层。

推理引擎为什么重要

层次	推理引擎要处理的问题
请求调度	不同用户、不同上下文长度、不同 batch 如何高效排队。
KV Cache	长上下文和多轮对话中，缓存如何复用、搬运和释放。
模型形态	自回归、扩散、flow matching 等不同推理过程有不同系统需求。
成本结构	延迟、吞吐、显存、硬件利用率共同决定产品能否规模化。
生态适配	上层模型变化快，下层硬件变化慢，中间层必须持续吸收两边变化。

这一段最值得保留的，不是 SGLang 的具体技术细节，而是赵晨阳对“中间层”的判断。AI 时代越往上层越热闹，越容易被产品叙事占满；但真正决定应用能否规模化的，往往是中间层的成本、吞吐和工程可靠性。推理引擎不像模型发布那样容易制造新闻，也不像应用那样直接面对用户，却会成为整个 AI 经济能否跑通的重要地基。

1.1 本章小结

SGLang 所在的推理引擎层，是 AI 应用、模型和硬件之间的中间层。它不直接回答“模型有多聪明”，而回答“模型服务如何以可承受成本稳定运行”。赵晨阳的经历之所以适合讨论退学潮，是因为他参与的不是抽象论文项目，而是一个处在产业成本结构核心位置的开源系统。

2 退学潮：不是反学校，而是 applied research 被迫面对市场

退学是这期节目最显眼的话题，但赵晨阳和主持人都没有把它包装成青春叛逆故事。赵晨阳说自己读了十五个月博士，这十五个月与做 SGLang 的时间高度重合。后来觉得 timing 正确，工业界机会比学术界更多，于是选择离开。这个决定背后的关键词不是“退学”，而是“什么样的 research 应该在哪里发生”。

他把 research 分成 applied research 和 fundamental research。Applied research 既然叫 applied，就必须有 application，必须真正接受市场、用户、工程系统和商业价值的检验。Fundamental research 则更适合学校，因为它可以暂时不在乎市场结果，而追求内在哲学体系的自洽和长期问题意识。问题在于 LLM 时代资源门槛抬高，即使是 fundamental research，要验证很多假设也需要大量算力和工程资源，学界越来越难提供。

赵晨阳对 applied research 的判断

Applied research 最大的点就是要有 application。若一个系统方向最终要靠大厂 adoption、真实用户和商业效果来证明价值，那么在学校里虚空 formulate 一个 evaluation，可能反而不如直接到工业界接受真实反馈。

这里还有一个地域差异。赵晨阳认为，美国读博的中国学生会更挣扎，因为签证和工作许可限制让他们很难在读博期间进入最前沿公司，接触最核心技术。相比之下，国内很多模型公司愿意给非正式编制的实习生更高权限；香港、新加坡或国内读博也不一定有同样强烈的离开必要。退学潮因此不是一种普遍建议，而是某些地区、某些身份、某些技术窗口叠加后的结果。

主持人补充了“离经叛道”的文化感：在中国传统观念里，退学常被视为不光彩；而硅谷创业叙事里，退学反而可能被理解成 outlier 特质。赵晨阳也承认，愿意退学的人往往更不循规蹈矩，这种特质对创业和投资人都有吸引力。但他并没有把退学神化。很多人留在学校也完全合理，关键在于你相信的问题、资源条件和机会窗口是否仍然匹配学校环境。

不要把退学潮误读成“读博无用”

这期节目真正讨论的不是“该不该退学”，而是不同类型 research 的最优承载环境。Fundamental research 仍然需要学校，系统型 applied research 若必须经受真实 adoption 和商业反馈，则可能更适合工业界。

尚晏仪引用戴雨森的意思，说创业需要某种情绪和信念。如果你非常冷静地推演终局，可能永远不会出来创业，因为总能想到大厂会吃掉你、上下游会挤压你、scope 会越收越小。创业不是完全理性规划出来的，它需要某种相信、冲动和愿意承担不确定性的能力。这也解释了为什么年轻人反而可能更敢跳出来：他们没有被长周期体制完全固定，还保留对 1% 机会的偏好。

2.1 本章小结

退学潮不是反学校，而是 AI 时代 applied research 与工业资源、真实反馈、市场验证之间的关系变得更直接。赵晨阳的选择说明，若一个方向本质上要通过工程 adoption 和商业价值证明，那么工业界可能比学校更本质；但 fundamental research、长期理论问题和自洽体系仍然有学校价值。退学不是模板，而是时机、身份、资源与问题类型共同作用的结果。

3 AI 时代的 Research Taste：人类剩下的是抽象层级

对谈中段从退学转向一个更大的问题：当 AI 产出越来越多 artifacts，人类如何判断什么 research 是好的？赵晨阳和主持人都承认，applied research 还可以用 revenue、用户价值或明确 benchmark 来反映；fundamental research 更难，因为它更像艺术，需要 taste 的人来判断。AI 时代 artifacts 增多，人类 review 带宽有限，传统 peer review 很难承受这样的压力。

这里出现一个关键判断：科研品位体现在对问题的抽象层级。一个具体问题背后可能有更 fundamental 的结构，例如 intelligence 里的 consensus、一致性、泛化、长期学习等。真正不容易被 AI 很快替代的研究，往往不是六个月 build 一个 soon-to-be-obsolete 系统，而是能够把具体 testbed 背后的长期问题抽象出来。

Research taste 的位置

在执行成本下降后，人类的價值不是比 AI 更快写更多代码或更多实验，而是提出更高层级的问题抽象，判断哪些问题值得做、哪些 tradeoff 重要、哪些路线只是局部优化。

赵晨阳提到自己喜欢写 debug log，并维护一个 machine learning systems tutorial。他期待未来 AI 能从这些 debug log 中学到试错过程。这个细节很重要，因为学术论文通常只呈现最后打磨好的 idea，很少记录中间的 tradeoff、失败路径和调试判断。恰恰这些“过程中的判断”，可能是 taste 难以被蒸馏、也难以被 AI 取代的原因。

这和谢赛宁访谈里“研究非线性”的观点形成呼应：论文写出来时看似线性，但真正的 idea 来自探索、失败、转向和重新理解问题。若这些过程不被记录，AI 只能学到结果形式，很难学到研究者为何放弃某条路、为何保留某个设计、为何某个失败其实很有信息量。

为什么 debug log 重要

- 它记录了论文不会写的失败路径。
- 它暴露设计 tradeoff，而不是只呈现最终选择。
- 它把“品位”从不可说的直觉，部分转化为可学习材料。
- 它可能成为未来 AI 学习科研过程的重要数据形态。

这一段还点出了一个悖论：AI 让执行廉价之后，好的 design 更重要。写代码的治理成本远低于雇人执行，前提是 design 足够清楚。人类的职责不再是把每一行代码亲手写出来，而是决定架构、边界、优先级、失败处理和长期目标。

3.1 本章小结

AI 时代 research taste 的核心不是“写得更多”，而是“抽象得更高、判断得更准、过程记录得更完整”。Applied research 可以靠市场反馈，fundamental research 更依赖 taste 与长期回看。debug log、tradeoff 记录和失败路径，可能成为未来 AI 学习研究品位的重要材料。

4 AI Coding：代码便宜之后，责任变贵了

AI Coding 是这期节目里最现实的一条线。主持人提到，Cloud Code 等工具让代码生产速度远超人类，GitHub reviewer、开源 maintainer 和公司内部 code review 都会被冲击。赵晨阳在 SGLang 内部已经面对 AI 生成代码的压力，因此他的回答非常具体。

他并不反对 AI 写代码，甚至欢迎 AI 写代码。关键是：提交 PR 的人必须能解释 design feature，必须愿意为结果负责。如果一个人交了 PR，自己都说不清楚细节和设计动机，这种 PR 可以毫无顾忌地 close 掉。AI 不是替你承担责任的代理；你用 AI 写，出事仍然要你负责。

开源项目面对 AI PR 的核心规则

欢迎 AI 写 code，但不欢迎不动脑子。贡献者必须解释设计、理解细节、愿意负责。AI 降低的是生产代码的成本，不是维护项目质量的责任。

这个判断非常有现实意义。Amazon 的例子也被提到：可以用 AI 写，但出了事要负责。责任机制会反过来抑制“每天生成 50K 代码”的冲动。因为 AI 生成越多，出错概率和 review 成本也越高。如果贡献者只是想刷 PR、刷贡献记录，而不愿为代码承担长期维护责任，开源项目会被冲烂。

主持人提出一个很关键的观察：Git 管理的是最终代码结果，不管理思考过程。我们看到 final diff，却看不到这个设计如何形成、模型如何推理、用户如何与 coding agent 交互、哪些方案被放弃。这与前面 debug log 的问题相连：AI Coding 时代真正稀缺的，不只是代码本身，而是设计过程、思考水平和责任链。

Git 记录代码，不记录判断

AI 时代的代码治理不能只看 diff 大小、PR 数量或代码量。两个 PR 代码量相同，背后的思考、风险和维护责任可能完全不同。未来真正需要记录的，是 decision flow、review 过程和设计理由。

这里引出 context graph 和 SaaS 的类比。SaaS 真正值钱的不只是 materialized data，还包括数据如何进入系统、背后的业务流程和 offline discussion。Coding 也类似：最终代码只是结果，背后的设计流、交互过程和上下文才是更深层资产。谁能捕捉这些过程，谁就可能在 AI Coding 时代形成新的数据与能力闭环。

4.1 本章小结

AI Coding 让代码生产变得便宜，但让责任、设计和 review 变得更重要。开源项目不能靠禁止 AI 代码解决问题，而要建立责任边界：贡献者必须理解、解释并维护自己的 PR。未来代码协作的关键数据，可能不只是 final diff，而是 decision flow、context graph 和设计理由。

5 AI 是放大器：从学习反馈到认知卸载

赵晨阳说 AI 是放大器：它放大人的主观诉求和主观能动性。这个判断有两面。对主动学习者来说，AI 是天大的福报。比如高中时想理解洛必达法则，传统教学可能只说“超纲不能用”，却不认真解释何时能用、为何能用；AI 可以快速回应求知欲，给出即时反馈，帮助人深入探索。

但尚晏仪提醒，要警惕认知卸载。如果一个人还没有想清楚自己要学什么、做什么，就把问题抛给 AI，让 AI 给多轮交付，长期会丧失主动思考能力。AI 反馈非常及时、甚至会哄人，图片和视频反馈又更刺激，很容易迅速建立多巴胺通路。她提到自己玩 AI 视频生成时感到这种反馈通路建立得非常快，因此觉得可怕。

AI 作为放大器，不自动放大好东西

AI 会放大主观能动性，也会放大懒惰、冲动和短反馈成瘾。对于已经有清晰目标的人，它是学习加速器；对于没有目标和判断力的人，它可能成为认知卸载工具。

Sora 下架和视频生成商业模式的讨论，延续了这一点。赵晨阳认为 Sora 技术并不差，问题可能在于视频推理太贵，而 OpenAI 自己没有视频生态能把生成内容融入 revenue 循环。快手或字节这类本来就有视频生态的平台，生成视频可以进入自己的分发和变现闭环；OpenAI 做一个独立视频产品，则会面对成本与收入不匹配。

这一段其实把“AI 是放大器”从个人学习扩展到商业生态。AI 视频会放大用户娱乐欲望，也会放大平台分发能力；但若没有生态闭环，技术强也未必是好生意。相同技术在不同组织结构中会产生完全不同的商业结果。

放大器逻辑的三层

层次	放大什么
个人学习	放大求知欲、反馈速度和自我训练能力，也可能放大认知卸载。
代码生产	放大执行速度，也放大低质量 PR 和 review 压力。
平台生态	放大内容生成和分发能力，但前提是成本能进入商业闭环。

5.1 本章小结

AI 不是单纯“更聪明的工具”，而是放大器。它会放大主动学习者的探索，也会放大不思考者的依赖；会放大代码生产，也会放大责任缺口；会放大视频生成，也会放大平台成本和内容刺激。关

键不是能不能用 AI，而是使用者有没有目标、审美和边界。

6 Agent 不会 Say No：人类的决策边界

对谈后半段有一个非常重要的判断：Agent 当前很难帮助人决定“不做什么”。尚晏仪举产品创业为例：目标是做成 profitable company，这个目标相对可量化，但行动路径无限，环境不断变化。每天都可以做很多事，竞品、技术、用户需求都在变。Agent 可以帮忙执行，却很难帮人决定优先级，尤其是决定哪些需求不做。

赵晨阳用一句话压缩了这个问题：AI 不 say no，因为它的 reward 是满足人类需求。但人的需求未必合理。不是每一个业务扩张都会给公司带来收益，很多需求长期看是负收益。SGLang 里绝大多数需求都会被无情关掉，因为维护者要判断什么对长期正确。

Say No 是人类决策能力的核心

AI 擅长满足已表达需求，但产品和工程常常需要拒绝需求。决定做什么重要，决定不做什么同样重要。边界、优先级和长期判断，是当下 Agent 最难替代的人类职责。

这和前面 AI Coding 的责任问题连在一起。人如果做了错误决定，AI 会更快执行错误决定。执行越便宜，错误方向的放大也越快。因此人的审美、边界把控和决策能力反而更重要。AI 时代不是人类退出，而是人类从执行层退到设计层、决策层、责任层。

主持人还把这个问题连接到 context graph 和 decision flow：如果未来要训练 Agent 学会更好的产品判断，就必须把人类如何做决策的数据留下来。但今天很多关键讨论发生在线下、会议、即时消息和组织语境中，很难被完整捕捉。最终代码或最终产品只是结果，真正重要的“为什么这样做、为什么不那样做”常常消失了。

6.1 本章小结

Agent 当前最大的短板之一，不是不会执行，而是不会拒绝。它很难理解长期目标下的优先级、资源约束、机会成本和战略边界。AI 让执行更快，因此人的决策错误也会被更快放大。未来值得记录和训练的不只是代码结果，而是 decision flow 和 say no 的理由。

7 开源社区：AI 时代如何记录贡献与维持秩序

最后一段回到开源社区。SGLang 作为广受关注的开源项目，面临的问题不是“有没有人贡献”，而是不同阶段社区需要不同治理方式。早期可能欢迎各种功能请求；项目成熟后，必须有人拍板，决定什么支持、什么不支持。否则项目会被无边需求拖垮。

AI 生成代码让这个问题更尖锐。赵晨阳的标准很明确：欢迎 AI 写 code，但贡献者必须动脑子，必须能解释设计细节，必须愿意负责。刷 PR、刷贡献、一天交二十个 PR，却不愿理解和维护，这对开源项目是伤害。真正负责任的贡献者，哪怕使用 AI，也会把 AI 当作执行工具，而不是责任外包工具。

主持人把问题扩展到社区本身：如何维护一个开源社区，如何减少信息差，如何让学生和年轻工程师不仅接收信息，还能一起做项目。社区的基础功能是信息传递，进阶功能是组织协作。meetup、群聊、开源项目、评测项目都可以成为共同做事的载体。

开源社区的三层功能

层次	作用
信息传递	让新人更快知道领域在发生什么，减少信息差。
协作组织	让有能力的人围绕项目、评测、工具和活动一起做事。
贡献沉淀	记录谁做了什么、贡献质量如何、非代码贡献如何被看见。

结尾处一个很有意思的问题，是如何记录每个人的贡献。代码贡献相对容易量化，但也不完全本质：同样代码量可能由不同模型生成，质量和思考量完全不同；非代码贡献如组织 meetup、写文档、做评测、传播信息、帮助新人，则更难记录。主持人提到想做小工具，把贡献沉淀和自动化记录起来。这其实和整场对谈的主线一致：AI 时代最难记录的，不是产出物，而是过程、判断和关系。

贡献数量不是贡献质量

AI 让贡献数量更容易被制造，因此社区更不能只看 PR 数、代码行数或提交频率。真正要衡量的是设计理解、长期维护、问题判断、协作质量和对社区信息流的贡献。

7.1 本章小结

开源项目在 AI 时代需要更强的治理，而不是更宽松的合并。AI 生成代码可以被接受，但责任不能被外包。社区的价值不只是代码仓库，还包括信息传递、协作组织、贡献记录和新人才培养。未来的开源基础设施，也许会围绕“如何记录过程与责任”重新设计。

8 总结：AI 降低执行成本，也提高判断成本

整场对谈可以收束成一句话：AI 正在快速降低执行成本，但这并没有让人类职责消失，反而让判断、责任、审美和组织能力变得更贵。赵晨阳的经历提供了一个很好的切面：他从学校走到 SGLang，再走向推理引擎创业，面对的不只是技术机会，也是 applied research、工程 adoption、代码治理和开源社区的责任边界。

问题意识	对谈中的核心判断
推理引擎	它处在应用、模型和硬件之间，是 AI 规模化运行的中间层。
退学潮	本质不是反学校，而是 applied research 更需要真实应用、资源和市场反馈。
Applied vs Fundamental	Applied research 要有 application；fundamental research 要有自洽的问题体系。
Research taste	AI 时代人类更需要高层抽象、问题选择和 tradeoff 判断。
Debug log	论文隐藏试错过程，debug log 可能保存 AI 难以学习的研究品位。
AI Coding	代码生产便宜后，design、review、责任和 say no 更重要。
认知卸载	AI 是放大器，能放大求知欲，也能放大懒惰和短反馈成瘾。
Agent 决策	Agent 不会自然 say no，人类仍要决定什么不做。
Decision flow	最终代码和最终产品只是结果，过程中的判断数据更稀缺。
开源社区	AI 生成代码时代，开源需要记录贡献、筛选责任、维护项目秩序。

如果说上一代软件工程的核心问题是“如何把人类写的代码组织起来”，那么 AI Coding 时代的问题会变成“如何把 AI 放大的产出、人的判断、责任边界和社区协作重新组织起来”。赵晨阳这期对谈的价值就在这里：它没有把 AI 描述成简单替代人类的力量，而是展示了替代发生之后，那些更难替代的东西如何浮出水面。

附录：关键判断索引

关键判断	所对应的问题意识
<i>Applied research</i> 最大的点就是要 有 <i>application</i> 。	这是退学和工业界选择的核心理由：如果研究最终靠真实 adoption 证明，工业环境可能更本质。
学术圈很多时候是在拿着锤子找 钉子。	这批评虚空 formulate evaluation 的惯性，也解释为什么 很多系统工作需要真实用户和真实工程反馈。
科研品位体现在对问题的抽象层 级。	AI 可以更快执行具体任务，人类更稀缺的是发现长期问 题和抽象结构。
我喜欢写 <i>debug log</i> 。	Debug log 记录论文不会写的试错、tradeoff 和判断过程， 可能是未来 AI 学习研究品位的重要材料。
AI 是放大器。	AI 会放大人的主观能动性，也会放大认知卸载、短反馈 成瘾和低质量产出。
AI 不 <i>say no</i> 。	Agent 会倾向满足需求，但产品、工程和创业经常需要拒 绝需求。边界判断仍是人的职责。
欢迎 AI 写 <i>code</i> ，但你要能解释 并负责。	这是开源项目面对 AI 生成代码的治理底线：责任不能外 包给模型。
<i>Git</i> 管的是最终代码，不管思考过 程。	AI Coding 时代真正稀缺的数据，可能是 decision flow、 设计理由和交互过程。
社区不只是信息传递，还要一起 做事。	开源社区的下一步，是从减少信息差走向组织项目、记录 贡献和沉淀协作。